ByteMart Digital

THE INTERVIEW PREP SERIES

Java Interview Q&A

Core language, OOP, exceptions, collections, generics, concurrency, JVM & coding

76 interview questions with concise, accurate model answers, including 10 hands-on coding challenges with working solutions — organized by topic for focused preparation.

76	10	12	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Language Fundamentals 4

Object-Oriented Programming 9

Exceptions 14

Collections Framework 17

Generics 21

Concurrency & Multithreading 23

Functional Programming & Streams 27

JVM & Memory 30

Strings & Immutability 32

Modern Java (8 to 21) 34

Classes, Nesting & Reflection 37

Coding Challenges 39

This bank contains 76 Java interview questions across 12 sections, including 10 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Language Fundamentals

Q1 What is the difference between == and equals()?

ANSWER

`==` compares **references** for object types — it asks whether two variables point to the exact same object in memory. `equals()` compares **logical equality**, and its behaviour is defined by how a class overrides it. For primitives, `==` compares the actual values directly, since primitives are not objects and have no reference.

By default, `Object.equals()` falls back to reference comparison (the same as `==`). Classes like `String`, `Integer`, and the collection classes override `equals()` to compare contents, which is almost always what you want when comparing objects.

```java
String a = new String("hi");
String b = new String("hi");
System.out.println(a == b);        // false — two distinct objects
System.out.println(a.equals(b));   // true  — same character content

int x = 5, y = 5;
System.out.println(x == y);        // true  — primitive value compare
```

WATCH OUT

A frequent bug is comparing objects with `==` and getting occasional 'correct' results due to caching or literal pooling, which then fails in production. For objects, always use `equals()` unless you deliberately want identity comparison.

Q2 Why are Strings immutable in Java, and what is the String pool?

ANSWER

A `String` object cannot be changed after it is created — methods like `concat`, `replace`, and `toUpperCase` return a brand-new `String` rather than modifying the original. Immutability is enforced because the internal character array is private and final, and the class itself is final so it cannot be subclassed to break the guarantee.

Immutability brings several benefits. Strings are automatically **thread-safe** (no synchronization needed to share them), they can be used as reliable **hash keys** because their hash code never changes, and they can be safely **cached and pooled**. The **string pool** is a special area of the heap where string literals are interned — identical literals refer to the same pooled object, saving memory.

```java
String a = "hi";                  // pooled literal
String b = "hi";                  // same pooled object
System.out.println(a == b);       // true

String c = new String("hi");      // explicitly a new object (not pooled)
System.out.println(a == c);       // false
String d = c.intern();            // returns the pooled reference
System.out.println(a == d);       // true
```

Because every modification creates a new object, building a string inside a loop with + is wasteful — it allocates a new `String` each iteration. Use `StringBuilder`, which mutates an internal buffer in place, for repeated concatenation.

> **KEY IDEA**
> Immutability gives thread safety, safe hashing, and literal pooling; reach for StringBuilder whenever you build a string in a loop.

Q3 Explain final, finally, and finalize.

ANSWER

These three share a prefix but are entirely unrelated — a classic trick question that tests whether you actually understand each.

- **final** is a modifier. A final **variable** can be assigned only once; a final **method** cannot be overridden; a final **class** cannot be extended. It is also required for variables captured by lambdas (effectively final).
- **finally** is a block that runs after a `try/catch`, whether or not an exception occurred, and is used for cleanup such as closing resources.
- **finalize()** was a method the garbage collector called before reclaiming an object. It is **deprecated** (since Java 9) and unreliable — there is no guarantee when or whether it runs.

```java
final int MAX = 100;              // constant, cannot be reassigned

try {
    risky();
} finally {
    cleanup();                    // always runs
}
```

> **TIP**
> For deterministic cleanup, use **try-with-resources** with `AutoCloseable` rather than relying on the deprecated `finalize()`.

Q4 What is the difference between checked and unchecked exceptions?

ANSWER

Checked exceptions extend `Exception` (but not `RuntimeException`) and represent conditions a program can reasonably be expected to recover from — a missing file, a failed network call. The compiler **forces** you to either catch them or declare them with `throws`.

Unchecked exceptions extend `RuntimeException` and usually indicate programming bugs — a null dereference, an out-of-bounds index, an illegal argument. The compiler does not force handling, because these should be fixed in code rather than caught. `Error` (like `OutOfMemoryError`) sits separately and should not be caught at all.

	Checked	Unchecked
Extends	Exception	RuntimeException
Compiler	must handle or declare	no requirement
Represents	recoverable conditions	programming errors
Examples	IOException, SQLException	NullPointerException, IllegalArgument

Use checked exceptions when the caller can sensibly react (retry, prompt the user), and unchecked when the situation reflects a bug that should be fixed rather than handled at runtime.

Q5 Explain autoboxing, unboxing, and the Integer cache pitfall.

ANSWER

Autoboxing is the automatic conversion of a primitive to its wrapper object (`int` to `Integer`), and **unboxing** is the reverse. It happens transparently in assignments, method calls, and collections, which only store objects.

The pitfall: Java caches `Integer` objects for values from −128 to 127. Two autoboxed `Integer`s in that range are the same cached object, so `==` appears to work — but for values outside the range, new objects are created and `==` returns false.

```java
Integer a = 100, b = 100;
System.out.println(a == b);       // true  — both from the cache
Integer c = 200, d = 200;
System.out.println(c == d);       // false — distinct objects
System.out.println(c.equals(d));  // true  — always use equals for wrappers
```

WATCH OUT

Unboxing a `null` wrapper throws `NullPointerException` at runtime — e.g. assigning a `null Integer` to an `int`. This is one of the most common sources of surprise NPEs, especially with values pulled from maps or databases.

Q6 What are the primitive types, and how does numeric promotion and casting work?

ANSWER

Java has eight primitives: `byte`, `short`, `int`, `long` (integers), `float`, `double` (floating point), `char` (a Unicode character), and `boolean`. Unlike reference types, they hold values directly, not references.

In arithmetic, smaller integer types are **promoted** to at least `int` before the operation. Assigning a wider type to a narrower one requires an explicit **cast**, which may truncate or overflow. Widening (e.g. `int` to `long`) is implicit and safe; narrowing needs a cast and can lose data.

```java
byte a = 10, b = 20;
// byte c = a + b;         // won't compile — a + b is promoted to int
byte c = (byte)(a + b);      // OK with an explicit cast

long big = 100;              // widening, automatic
int small = (int) big;       // narrowing, needs a cast
```

Q7 Is Java pass-by-value or pass-by-reference?

ANSWER

Java is **always pass-by-value**. For primitives, the value itself is copied into the method. For objects, the reference (the pointer to the object) is copied by value — so the method receives a copy of the reference, not the caller's variable.

This distinction matters: a method can mutate the object a reference points to (visible to the caller), but reassigning the parameter to a new object does **not** affect the caller's variable, because the method only holds a copy of the reference.

© 2026 ByteMart Digital

```java
void modify(int[] arr, int x) {
    arr[0] = 99;    // affects the caller — same underlying array
    x = 5;          // local copy only — no effect on the caller
}
// after modify(a, n): a[0] is 99, but n is unchanged
```

KEY IDEA

Java copies references by value: methods can change the object's internals but cannot reseat the caller's variable to a different object.

Q8 What is the difference between static and instance members?

ANSWER

A **static** member belongs to the class itself — there is one copy shared by all instances, accessed via the class name. An **instance** member belongs to each object — every instance has its own copy, accessed through an object reference.

A static method has no `this` reference and therefore cannot directly access instance fields or call instance methods. Static members are initialized once when the class is first loaded; instance members are initialized each time an object is created.

WATCH OUT

Trying to reference an instance field from a static method (such as `main`) is a compile error the exam and interviewers plant deliberately. Static context has no object to bind `this` to.

© 2026 ByteMart Digital

SECTION 02

Object-Oriented Programming

Q9 What are the four pillars of OOP?

ANSWER

Object-oriented programming rests on four principles that together make code modular, reusable, and maintainable.

- **Encapsulation** — bundling data with the methods that operate on it, and hiding internal state behind a controlled interface (private fields, public methods).
- **Inheritance** — deriving a class from another to reuse and extend its behaviour, expressing an 'is-a' relationship.
- **Polymorphism** — treating objects of different types through a common interface, with the actual method resolved at runtime.
- **Abstraction** — exposing only the essential features of something while hiding the implementation details behind interfaces or abstract classes.

> **KEY IDEA**
> Encapsulation hides state, inheritance reuses behaviour, polymorphism enables one interface for many types, and abstraction hides complexity.

Q10 When would you use an interface versus an abstract class?

ANSWER

Use an **interface** to define a capability or contract that unrelated classes can implement, and to gain multiple inheritance of type — a class can implement many interfaces. Since Java 8, interfaces can also carry `default` and `static` methods with bodies.

Use an **abstract class** when a group of related classes share common state and partial implementation. It can hold fields, constructors, and concrete methods, but a class can extend only one abstract class.

	Interface	Abstract class
State	constants only	instance fields allowed
Inheritance	implement many	extend one
Constructor	no	yes
Use for	capability / contract	shared base with state

© 2026 ByteMart Digital

TIP
A useful rule of thumb: interfaces model what a type can do (a capability), abstract classes model what a type is (a partial base implementation).

Q11 What is the difference between overloading and overriding?

ANSWER

Overloading is a compile-time concept: multiple methods in the same class share a name but differ in their parameter lists (number, types, or order). The compiler picks which one to call based on the arguments. Return type alone does not distinguish overloads.

Overriding is a runtime concept: a subclass provides a new implementation of a method inherited from its superclass, with the same signature. The version that runs is determined by the object's actual type through dynamic dispatch — the essence of polymorphism.

```java
class Printer {
    void print(int x)    { }        // overloaded
    void print(String s) { }        // overloaded (different param)
}
class Base { String name() { return "Base"; } }
class Sub extends Base {
    @Override String name() { return "Sub"; }   // overridden
}
```

WATCH OUT
A method with a different parameter list is **overloading**, not overriding. Always use `@Override` so the compiler catches an accidental overload where you intended to override.

Q12 How do polymorphism and dynamic method dispatch work?

ANSWER

Polymorphism lets a superclass reference point to a subclass object. Which members are visible at compile time is decided by the reference type, but which overridden method actually runs is decided at runtime by the object's real type — this is **dynamic dispatch**.

```java
Animal a = new Dog();   // upcast — always safe
a.sound();              // runs Dog's override at runtime
// a.fetch();           // won't compile — not visible on Animal
```

To call subclass-specific members through a superclass reference, you must downcast, guarded by `instanceof` to avoid a `ClassCastException`. Polymorphism is what lets you write code against a general type and have the right specific behaviour execute automatically.

Q13 What is the contract between equals() and hashCode()?

ANSWER

The contract has two core rules. First, if two objects are equal according to `equals()`, they **must** return the same `hashCode()`. Second, unequal objects may share a hash code (a collision), but for good performance they should be well distributed.

This matters because hash-based collections (`HashMap`, `HashSet`) use `hashCode()` to find the right bucket and then `equals()` to confirm the match. If the contract is broken, a key you inserted can become unfindable.

java

```
@Override public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User u)) return false;
    return Objects.equals(id, u.id);
}
@Override public int hashCode() {
    return Objects.hash(id);          // same field(s) as equals
}
```

WATCH OUT

Whenever you override `equals()` you must override `hashCode()` using the same fields — otherwise equal objects can land in different buckets and hash lookups fail.

Q14 What is the difference between composition and inheritance?

ANSWER

Inheritance creates an 'is-a' relationship — a subclass inherits and extends a superclass. **Composition** creates a 'has-a' relationship — a class holds references to other objects and delegates work to them.

Composition is generally preferred ('favour composition over inheritance') because it is more flexible and avoids the fragility of deep inheritance hierarchies, where a change in a base class can unexpectedly break subclasses. Inheritance tightly couples subclass to superclass; composition keeps components independent and swappable.

© 2026 ByteMart Digital

```java
// composition — Car HAS-A Engine, delegates to it
class Car {
    private final Engine engine;
    Car(Engine engine) { this.engine = engine; }
    void start() { engine.start(); }
}
```

TIP

Use inheritance only for genuine 'is-a' relationships with stable base behaviour; reach for composition when you want to reuse behaviour flexibly.

Q15 What are access modifiers, and what does 'default' access mean?

ANSWER

Access modifiers control the visibility of classes and members. There are four levels, and the subtle one is **default** (no keyword), which means package-private.

Modifier	Visible to
private	the same class only
(default)	the same package
protected	the same package and subclasses
public	everywhere

WATCH OUT

'Default' access is not the same as `protected`: a package-private member is **not** visible to a subclass in a different package, whereas a `protected` one is. Interviewers probe exactly this gap.

Q16 What is the difference between an abstract class and abstract method, and their rules?

ANSWER

An **abstract class** cannot be instantiated; it exists to be extended. An **abstract method** has no body and forces subclasses to implement it. A class with any abstract method must itself be declared abstract, but an abstract class may also contain fields, constructors, and concrete methods.

- A method cannot be both `abstract` and `final` (final blocks overriding).
- A method cannot be both `abstract` and `private` or `static` (these cannot be overridden).
- An abstract class can have constructors, called via `super()` from subclasses.

- An abstract class need not have any abstract methods.

© 2026 ByteMart Digital

SECTION 03

Exceptions

Q17 How do try, catch, and finally interact?

ANSWER

The `try` block guards code that might throw. A matching `catch` handles a specific exception type; catch blocks are evaluated top to bottom, so more specific exceptions must come before more general ones. The `finally` block always runs — after normal completion, after a caught exception, and even after a `return` — making it ideal for cleanup.

```java
try {
    return compute();
} catch (ArithmeticException e) {
    return handle(e);
} finally {
    cleanup();          // runs regardless, even though we returned
}
```

WATCH OUT

A `return` (or `throw`) inside `finally` overrides any earlier return or exception — it can silently swallow the real result. Avoid returning from `finally`.

Q18 What is try-with-resources, and why prefer it?

ANSWER

Try-with-resources declares one or more resources in the `try` header; any resource implementing `AutoCloseable` is closed automatically when the block ends — in reverse order of declaration, and even if the body throws. It removes the boilerplate and bugs of manual `finally` cleanup.

```java
try (var reader = Files.newBufferedReader(path);
     var writer = Files.newBufferedWriter(out)) {
    writer.write(reader.readLine());
}   // both closed automatically, writer first
```

If both the body and a `close()` throw, the body's exception propagates and the close exception is attached as a **suppressed** exception, retrievable via `getSuppressed()` — so you never lose the original cause.

Q19 How do you create and use a custom exception?

ANSWER

Define a custom exception by extending `RuntimeException` (unchecked) or `Exception` (checked), providing constructors that pass a message and, ideally, a cause to the superclass. Custom exceptions make error handling domain-specific and readable.

```java
public class OrderNotFoundException extends RuntimeException {
    public OrderNotFoundException(Long id) {
        super("Order not found: " + id);
    }
}
```

> **TIP**
> Prefer unchecked custom exceptions for most application errors, and always preserve the original cause (`super(message, cause)`) so stack traces stay useful.

Q20 What is the difference between throw and throws?

ANSWER

`throw` is a statement that actually raises an exception object at runtime. `throws` is part of a method signature that declares the checked exceptions a method may propagate, informing callers they must handle or re-declare them.

```java
void load(String path) throws IOException {   // declares
    if (path == null)
        throw new IllegalArgumentException("null path");   // raises
    Files.readString(Path.of(path));
}
```

Q21 Explain the exception hierarchy.

ANSWER

At the root is `Throwable`, with two branches. `Error` represents serious problems the application should not try to handle (e.g. `OutOfMemoryError`, `StackOverflowError`). `Exception` represents conditions applications may catch.

Under `Exception`, `RuntimeException` and its subclasses are **unchecked**, while all other `Exception` subclasses are **checked**. Understanding this tree explains which exceptions the compiler forces you to handle.

SECTION 04

Collections Framework

Q22 How does a HashMap work internally?

ANSWER

A `HashMap` stores key-value entries in an array of **buckets**. When you put a key, its `hashCode()` is computed and spread (to reduce collisions), then mapped to a bucket index. Multiple keys landing in the same bucket form a linked list; since Java 8, a bucket that grows past a threshold converts to a balanced (red-black) tree for O(log n) lookups instead of O(n).

To find a key, the map goes to its bucket and compares candidates with `equals()`. When the number of entries exceeds the capacity times the load factor (default 0.75), the map **resizes** — doubling the bucket array and rehashing all entries.

Operation	Average	Worst (pre-Java 8)
get / put	O(1)	O(n)
with tree bins (Java 8+)	O(1)	O(log n)

KEY IDEA

hashCode picks the bucket, equals resolves collisions within it; a poor hashCode degrades a HashMap toward linear time.

Q23 What is the difference between ArrayList and LinkedList?

ANSWER

`ArrayList` is backed by a resizable array, giving O(1) random access by index but O(n) insertion or removal in the middle (elements must shift). `LinkedList` is a doubly-linked list, giving O(1) insertion and removal at the ends (or via an iterator) but O(n) random access.

Operation	ArrayList	LinkedList
get(index)	O(1)	O(n)
add/remove at end	O(1) amortized	O(1)
add/remove in middle	O(n)	O(n) (plus traversal)
memory	compact	extra node pointers

TIP

In practice `ArrayList` is the right default for the vast majority of cases; `LinkedList` only wins for heavy end-insertions or when used as a deque, and even then `ArrayDeque` is often better.

© 2026 ByteMart Digital

Q24 What is the difference between HashMap, ConcurrentHashMap, and Hashtable?

ANSWER

`HashMap` is not thread-safe and allows one null key. The legacy `Hashtable` synchronizes every method (locking the whole map), which is thread-safe but a bottleneck. `ConcurrentHashMap` is the modern thread-safe choice — it locks only small segments of the map, so many threads read and write concurrently with minimal contention.

	HashMap	Hashtable	ConcurrentHashMap
Thread-safe	no	yes (full lock)	yes (fine-grained)
Performance	fast	slow under load	fast under load
Null keys	one	none	none

WATCH OUT

Never share a plain `HashMap` across threads without external synchronization — concurrent structural changes can corrupt it or cause an infinite loop. Prefer `ConcurrentHashMap`.

Q25 What are fail-fast and fail-safe iterators?

ANSWER

A **fail-fast** iterator throws `ConcurrentModificationException` if the underlying collection is structurally modified during iteration (other than through the iterator itself). It works by tracking a modification count. Collections like `ArrayList` and `HashMap` are fail-fast.

A **fail-safe** iterator operates on a copy or snapshot of the data, so concurrent modification does not throw — but the iterator may not reflect the latest changes. `CopyOnWriteArrayList` and `ConcurrentHashMap` provide fail-safe iteration.

```java
// safe removal during iteration on a fail-fast list:
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (shouldRemove(it.next())) it.remove();   // via the iterator
}
```

Q26 What is the difference between HashSet, LinkedHashSet, and TreeSet?

ANSWER

All three implement `Set` (no duplicates) but differ in ordering and performance. `HashSet` is unordered and offers O(1) operations. `LinkedHashSet` maintains insertion order at a small cost. `TreeSet` keeps elements sorted (natural order or a comparator) with O(log n) operations, backed by a red-black tree.

	Ordering	Performance
HashSet	none	O(1)
LinkedHashSet	insertion order	O(1)
TreeSet	sorted	O(log n)

Q27 What is the difference between Comparable and Comparator?

ANSWER

`Comparable` defines a type's single **natural ordering** via `compareTo`, implemented by the class itself. `Comparator` is a separate object defining an **external** ordering — useful when you need multiple orderings or cannot modify the class.

```java
// natural ordering
record Item(String name, int price) implements Comparable<Item> {
    public int compareTo(Item o) { return name.compareTo(o.name); }
}
// external orderings, composed fluently
list.sort(Comparator.comparingInt(Item::price)
                    .thenComparing(Item::name).reversed());
```

Q28 How does HashMap resizing and the load factor work?

ANSWER

A `HashMap` starts with a default capacity (16 buckets) and a load factor (0.75). When the number of entries exceeds capacity times load factor (12 for the defaults), the map doubles its bucket array and rehashes every entry into the new, larger array.

Resizing is expensive, so if you know the approximate size in advance, construct the map with an appropriate initial capacity to avoid repeated rehashing. A lower load factor reduces collisions but uses more memory; the default 0.75 balances the two.

© 2026 ByteMart Digital

> **TIP**
> Pre-sizing with `new HashMap<>(expectedSize / 0.75 + 1)` avoids resize churn when inserting many known entries.

Q29 What is the difference between Iterator and ListIterator?

ANSWER

`Iterator` traverses a collection forward only and supports `remove()`. `ListIterator`, available on lists, adds **bidirectional** traversal (`hasPrevious`/`previous`), the ability to `add` and `set` elements during iteration, and index access.

SECTION 05

Generics

Q30 What are generics, and what problem do they solve?

ANSWER

Generics let you parameterise types — writing a class, interface, or method that works over a type supplied later. Their main benefit is **compile-time type safety**: the compiler catches type mismatches that would otherwise surface as runtime `ClassCastExceptions`, and they remove the need for explicit casts.

```java
List<String> names = new ArrayList<>();
names.add("Prakhar");
String first = names.get(0);   // no cast needed, type-checked
// names.add(42);              // compile error — caught early
```

KEY IDEA

Generics move type errors from runtime to compile time and eliminate manual casts, making collections and APIs safer and clearer.

Q31 Explain bounded types and wildcards (the PECS rule).

ANSWER

A **bounded type** constrains a type parameter, e.g. `<T extends Number>` accepts `Number` and its subtypes. **Wildcards** add flexibility to method parameters: `<? extends T>` is an upper bound (you can read T out), and `<? super T>` is a lower bound (you can write T in).

The mnemonic is **PECS** — Producer Extends, Consumer Super. Use `extends` when a structure produces values you read; use `super` when it consumes values you write.

```java
// producer — we read Numbers out of src
void copy(List<? extends Number> src, List<? super Number> dst) {
    for (Number n : src) dst.add(n);   // consumer — we write into dst
}
```

Q32 What is type erasure?

ANSWER

Java generics are implemented by **type erasure**: generic type information exists only at compile time and is removed (erased) in the bytecode, where type parameters become their bounds (usually `Object`). This keeps generics backward-compatible with pre-generics code.

Erasure has consequences: you cannot do `new T()`, cannot use `instanceof` with a parameterised type, and `List<String>` and `List<Integer>` share the same runtime class. It is also why you cannot overload two methods that differ only by generic parameter.

> **WATCH OUT**
> Because types are erased, generic type checks are compile-time only — raw-type or reflective code can still insert the wrong type and cause a `ClassCastException` later.

SECTION 06

Concurrency & Multithreading

Q33 What are the ways to create a thread, and why call start() not run()?

ANSWER

You can create a thread by implementing `Runnable` (preferred, since it leaves you free to extend another class) or extending `Thread`. In modern code you rarely manage raw threads — you submit tasks to an `ExecutorService` thread pool instead.

```java
Runnable task = () -> System.out.println("working");
Thread t = new Thread(task);
t.start();     // spawns a new thread, then calls run() there
// t.run();    // WRONG — runs on the current thread, no concurrency
```

WATCH OUT

Calling `run()` directly executes the code on the current thread with no new thread created — a classic interview trap. Only `start()` creates a new thread.

Q34 What is a thread's lifecycle?

ANSWER

A thread moves through well-defined states. **NEW** (created but not started), **RUNNABLE** (eligible to run, possibly running), **BLOCKED** (waiting for a monitor lock), **WAITING/TIMED_WAITING** (awaiting a signal or timeout), and **TERMINATED** (finished).

State	Meaning
NEW	created, not yet started
RUNNABLE	running or ready to run
BLOCKED	waiting to acquire a lock
WAITING / TIMED_WAITING	awaiting a signal / with timeout
TERMINATED	completed

Q35 Explain synchronized, volatile, and explicit Locks.

ANSWER

These address the two core concurrency problems: **visibility** (a change made by one thread being seen by others) and **atomicity** (an operation completing without interference).

- **volatile** — guarantees visibility of a single variable across threads and prevents certain reorderings, but does not make compound actions (like `count++`) atomic. Good for a simple flag.
- **synchronized** — provides both mutual exclusion (one thread in the block at a time) and visibility, using an object's intrinsic monitor lock.
- **Lock** (`ReentrantLock`) — an explicit, more flexible alternative to `synchronized` supporting `tryLock`, timed and interruptible waits, and fairness.

WATCH OUT
`volatile` alone cannot make `count++` safe — that read-modify-write is three steps. Use `synchronized` or an `AtomicInteger` for atomic counters.

Q36 What is a deadlock, and how do you prevent it?

ANSWER

A **deadlock** is a standstill where two or more threads each hold a lock the others need and none can proceed. It requires four simultaneous conditions (mutual exclusion, hold-and-wait, no preemption, and circular wait); breaking any one prevents it.

- Acquire locks in a consistent **global order** everywhere — breaks circular wait.
- Use `tryLock` with a timeout so a thread backs off instead of blocking forever.
- Minimise lock scope and hold time; avoid calling foreign code while holding a lock.
- Prefer higher-level concurrency utilities over hand-written locking.

Q37 What is the ExecutorService, and how do Callable and Future work?

ANSWER

An `ExecutorService` manages a pool of worker threads that you submit tasks to, decoupling task submission from thread management. It reuses threads (avoiding creation cost), bounds concurrency, and provides lifecycle control via `shutdown()`.

A `Runnable` returns nothing; a `Callable<T>` returns a value and may throw. Submitting a `Callable` returns a `Future<T>`, whose `get()` blocks until the result is ready (or an exception is raised).

```java
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Integer> f = pool.submit(() -> expensiveCompute());
// do other work...
Integer result = f.get();       // blocks for the result
pool.shutdown();
```

Q38 How do wait(), notify(), and notifyAll() work?

ANSWER

These methods coordinate threads using an object's monitor and must be called while holding that object's lock (inside a `synchronized` block). `wait()` releases the lock and suspends the thread until another thread calls `notify()` (wakes one waiter) or `notifyAll()` (wakes all) on the same object.

```java
synchronized (queue) {
    while (queue.isEmpty())      // always wait in a loop
        queue.wait();
    process(queue.poll());
}
```

WATCH OUT

Always call `wait()` inside a **while** loop that rechecks the condition, not an `if` — threads can wake spuriously, and the condition may no longer hold.

Q39 What is ThreadLocal, and when is it useful?

ANSWER

`ThreadLocal` gives each thread its own independent copy of a variable, so no synchronization is needed to share the holder. It is commonly used for per-thread context like date formatters, user/security context, or database connections in a request.

WATCH OUT

On thread pools, threads are reused, so a `ThreadLocal` not cleared with `remove()` can leak memory and leak data between unrelated tasks. Always clean up in a `finally`.

Q40 What are the atomic classes, and how do they help?

ANSWER

Classes in `java.util.concurrent.atomic` — such as `AtomicInteger` and `AtomicLong` — provide lock-free, thread-safe operations on single variables using low-level compare-and-swap (CAS) hardware instructions. They make counters and accumulators atomic without the overhead of locking.

```java
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();      // atomic, lock-free
int value = counter.get();
```

Q41 What is CompletableFuture, and how does it improve on Future?

ANSWER

`CompletableFuture` extends the idea of a `Future` with a fluent, non-blocking API for composing asynchronous tasks. Instead of blocking on `get()`, you attach callbacks (`thenApply`, `thenCompose`, `thenCombine`) that run when the result is ready, and you can handle errors with `exceptionally`.

```java
CompletableFuture
    .supplyAsync(() -> fetchUser(id))
    .thenApply(User::name)
    .thenAccept(System.out::println)
    .exceptionally(ex -> { log(ex); return null; });
```

SECTION 07

Functional Programming & Streams

Q42 What is a functional interface, and how do lambdas relate?

ANSWER

A **functional interface** has exactly one abstract method (a SAM type). Standard examples include `Runnable`, `Predicate<T>`, `Function<T,R>`, `Supplier<T>`, and `Consumer<T>`. A **lambda expression** is a concise way to provide that single method's implementation.

The `@FunctionalInterface` annotation documents intent and makes the compiler reject a second abstract method. Lambdas make behaviour a first-class value you can pass to methods.

```java
Predicate<String> nonEmpty = s -> !s.isEmpty();
Function<String, Integer> length = String::length;
Supplier<List<String>> factory = ArrayList::new;
```

Q43 What are method references, and what kinds exist?

ANSWER

A **method reference** is a shorthand for a lambda that just calls an existing method. It reads better and signals intent clearly. There are four kinds: static (`Integer::parseInt`), bound instance (`obj::method`), unbound instance (`String::toUpperCase`), and constructor (`ArrayList::new`).

```java
list.forEach(System.out::println);          // bound instance
list.stream().map(String::toUpperCase);     // unbound instance
stream.map(Integer::parseInt);              // static
```

Q44 Explain the difference between intermediate and terminal stream operations.

ANSWER

A stream pipeline has a source, zero or more **intermediate** operations, and one **terminal** operation. Intermediate operations (`filter`, `map`, `sorted`, `distinct`) are **lazy** — they build up the pipeline and return a new stream but do no work yet. The terminal operation (`collect`, `forEach`, `reduce`, `count`) triggers execution and produces a result or side effect.

© 2026 ByteMart Digital

```java
List<String> result = people.stream()
    .filter(p -> p.age() >= 18)     // intermediate (lazy)
    .map(Person::name)              // intermediate (lazy)
    .sorted()                       // intermediate (lazy)
    .collect(Collectors.toList());  // terminal — runs the pipeline
```

WATCH OUT

Nothing runs until the terminal operation, and a stream is **single-use** — reusing one after its terminal op throws `IllegalStateException`.

Q45 What problem does Optional solve, and how should it be used?

ANSWER

`Optional<T>` is a container that may or may not hold a value, making the possibility of absence explicit in the type system rather than hidden behind a `null`. It encourages callers to handle the empty case and reduces `NullPointerExceptions`.

Prefer the functional accessors — `map`, `filter`, `orElse`, `orElseThrow`, `ifPresent` — over calling `get()`, which throws when empty and defeats the purpose.

```java
return repo.findById(id)
    .map(this::toDto)
    .orElseThrow(() -> new NotFoundException(id));
```

TIP

Use `Optional` as a return type for 'might not find one' methods; avoid it for fields and method parameters.

Q46 What is Collectors, and how does groupingBy work?

ANSWER

`Collectors` provides ready-made reduction operations for the terminal `collect` step — accumulating a stream into a list, set, map, joined string, or statistics. `groupingBy` partitions elements into a `Map` keyed by a classifier function, optionally with a downstream collector such as `counting` or `mapping`.

```java
Map<String, Long> byStatus = orders.stream()
    .collect(Collectors.groupingBy(Order::status,
                                   Collectors.counting()));
String csv = names.stream().collect(Collectors.joining(", "));
```

Q47 What is the difference between map and flatMap?

ANSWER

`map` transforms each element to exactly one output element (a one-to-one mapping). `flatMap` transforms each element into a stream of elements and then flattens all those streams into one — a one-to-many mapping. Use `flatMap` to flatten nested structures like a list of lists.

```java
// map: one-to-one
Stream<Integer> lengths = words.stream().map(String::length);
// flatMap: flatten a list of lists into a single stream
List<String> all = lists.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

SECTION 08

JVM & Memory

Q48 What is the difference between the heap and the stack?

ANSWER

The **stack** is per-thread and holds method call frames — local variables, parameters, and references. It grows and shrinks automatically as methods are entered and returned, and is fast and thread-confined. The **heap** is shared across all threads and holds every object; it is managed by the garbage collector.

A local variable of a reference type lives on the stack, but the object it points to lives on the heap. Excessive recursion exhausts the stack (`StackOverflowError`); too many live objects exhaust the heap (`OutOfMemoryError`).

	Stack	Heap
Scope	per thread	shared
Holds	frames, locals, refs	objects
Managed by	automatic (frames)	garbage collector

Q49 How does garbage collection work in Java?

ANSWER

The garbage collector automatically reclaims heap memory occupied by objects that are no longer **reachable** from any live reference (roots such as stack variables and static fields). You never free memory manually.

Modern collectors use a **generational** model based on the observation that most objects die young. The heap is divided into a young generation (collected frequently and cheaply) and an old generation (collected less often). Collectors like G1 and ZGC focus on keeping pause times low.

> **NOTE**
> You rarely tune GC by hand — do so only when profiling reveals a real problem, and prefer choosing an appropriate collector over micro-tuning flags.

Q50 What causes memory leaks in Java if there is a garbage collector?

ANSWER

GC only reclaims unreachable objects, so a 'leak' in Java means unintentionally keeping objects **reachable** longer than needed. Common causes include static collections that grow without bound, listeners or callbacks never unregistered, unclosed resources, and `ThreadLocal`s not removed on pooled threads.

```java
static final List<Object> CACHE = new ArrayList<>();
void handle(Object o) { CACHE.add(o); }   // never removed — grows forever
```

> **TIP**
> Bound caches (use `WeakHashMap` or an eviction policy), unregister listeners, close resources, and clear `ThreadLocal`s to prevent reachable-but-unused growth.

Q51 How does class loading work?

ANSWER

Classes are loaded on demand by **class loaders** in three phases: **loading** (reading the bytecode), **linking** (verification, preparation of static fields, and resolution), and **initialization** (running static initialisers). Loaders follow a **parent-delegation** model — a loader asks its parent before loading a class itself, which protects core classes from being overridden.

Q52 What is the JIT compiler?

ANSWER

The JVM starts by **interpreting** bytecode, then the **Just-In-Time (JIT)** compiler identifies hot code paths at runtime and compiles them to optimised native machine code. Because it optimises based on actual runtime behaviour (profiling), long-running Java programs can reach performance competitive with statically compiled languages after warm-up.

© 2026 ByteMart Digital

SECTION 09

Strings & Immutability

Q53 What is the difference between String, StringBuilder, and StringBuffer?

ANSWER

`String` is immutable — every modification creates a new object. `StringBuilder` and `StringBuffer` are mutable sequences designed for building strings efficiently. The difference between the latter two is thread safety: `StringBuffer`'s methods are synchronized (thread-safe but slower), while `StringBuilder` is not synchronized (faster, single-thread).

	String	StringBuilder	StringBuffer
Mutable	no	yes	yes
Thread-safe	yes (immutable)	no	yes (synchronized)
Use for	constants, keys	building (single thread)	building (shared)

KEY IDEA

Use `StringBuilder` for almost all string building; reach for `StringBuffer` only when the buffer is genuinely shared across threads.

Q54 What is the difference between final, immutable, and effectively final?

ANSWER

final means a variable cannot be reassigned — but if it references a mutable object, that object's contents can still change. **Immutable** means the object's state cannot change after construction (like `String`). **Effectively final** describes a variable that is never reassigned even without the keyword — required for variables captured by lambdas and anonymous classes.

```java
final List<String> list = new ArrayList<>();
list.add("ok");     // allowed — final ref, but the list is mutable
// list = other;    // not allowed — cannot reassign a final ref
```

Q55 What is the difference between shallow copy and deep copy?

ANSWER

A **shallow copy** duplicates an object but shares references to the nested objects it contains — changes to a nested object are visible through both copies. A **deep copy** recursively duplicates the nested objects too, producing a fully independent clone.

© 2026 ByteMart Digital

WATCH OUT

The default `Object.clone()` produces a shallow copy. For true independence, copy mutable nested objects yourself (e.g. via a copy constructor).

© 2026 ByteMart Digital

SECTION 10

Modern Java (8 to 21)

Q56 What are records, and what do they give you?

ANSWER

A **record** (Java 16+) is a concise, immutable data carrier. From a compact header, the compiler generates a canonical constructor, private final fields, accessor methods, and correct `equals`, `hashCode`, and `toString` — eliminating boilerplate for value types like DTOs.

```java
public record Point(int x, int y) { }
var p = new Point(3, 4);
p.x();                  // accessor
p.equals(new Point(3, 4));   // true — value-based
```

> **TIP**
> Records are ideal for DTOs, keys, and return types; they are implicitly final and cannot extend a class.

Q57 What are sealed classes?

ANSWER

A **sealed** class or interface (Java 17+) restricts which classes may extend or implement it, using a `permits` clause. This lets you model a closed set of subtypes, which the compiler can then check exhaustively — pairing well with pattern-matching switches.

```java
public sealed interface Shape permits Circle, Square { }
public record Circle(double r) implements Shape { }
public record Square(double s) implements Shape { }
```

Q58 What is an enum, and can it have fields and methods?

ANSWER

An **enum** defines a fixed set of named constants and is a full class — it can have fields, constructors, and methods, and each constant can even override behaviour. Enums are implicitly final and singleton per constant, which makes them a safe, expressive alternative to integer constants.

© 2026 ByteMart Digital

```java
enum Planet {
    EARTH(9.81), MARS(3.71);
    private final double gravity;
    Planet(double g) { this.gravity = g; }
    double gravity() { return gravity; }
}
```

Q59 What are default and static methods in interfaces?

ANSWER

Since Java 8, an interface can provide **default** methods (with a body) that implementing classes inherit without overriding, and **static** methods that belong to the interface itself. Default methods allow interfaces to evolve without breaking existing implementers.

```java
interface Greeter {
    String name();
    default String greet() { return "Hi, " + name(); }
}
```

WATCH OUT

If a class inherits conflicting defaults from two interfaces, it must override the method to resolve the ambiguity — the 'diamond' case.

Q60 What are switch expressions and pattern matching?

ANSWER

Modern **switch expressions** (Java 14+) return a value, use arrow labels with no fall-through, and can be exhaustive. Combined with **pattern matching** (Java 21), a switch can match on type and bind a variable in one step, replacing chains of `instanceof` and casts.

```java
String describe(Object o) {
    return switch (o) {
        case Integer i -> "int " + i;
        case String s  -> "string of length " + s.length();
        default        -> "other";
    };
}
```

© 2026 ByteMart Digital

Q61 What are varargs?

ANSWER

Varargs (`Type... name`) let a method accept a variable number of arguments, which the compiler packages into an array. There can be only one varargs parameter and it must be last. It underpins methods like `String.format` and `List.of`.

```java
static int sum(int... nums) {
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum(1, 2, 3);   // 6
```

Q62 What are the methods every class inherits from Object?

ANSWER

Every class implicitly extends `Object`, inheriting methods you frequently override or use: `equals` and `hashCode` (equality and hashing), `toString` (string form), `getClass` (runtime type), `clone` (copying), and the thread-coordination methods `wait`, `notify`, and `notifyAll`.

SECTION 11

Classes, Nesting & Reflection

Q63 What are the kinds of nested classes?

ANSWER

Java supports four nested class forms, each with a different relationship to the enclosing class.

- **Static nested class** — associated with the outer class, not an instance; used for grouping and helpers.
- **Inner (non-static) class** — tied to an outer instance and can access its members; holds an implicit reference to it.
- **Local class** — declared inside a method, visible only there.
- **Anonymous class** — an unnamed one-off implementation of an interface or class, often replaced today by a lambda.

WATCH OUT

A non-static inner class keeps a hidden reference to its outer instance, which can cause memory leaks if the inner object outlives its purpose. Prefer a static nested class when you do not need the outer instance.

Q64 What is reflection, and when would you use it?

ANSWER

Reflection lets code inspect and manipulate classes, methods, and fields at runtime — discovering types, reading annotations, and invoking members dynamically. Frameworks like Spring and Hibernate rely on it for dependency injection and ORM mapping.

WATCH OUT

Reflection is powerful but comes at a cost: it is slower, bypasses compile-time type checking, and can break encapsulation. Use it for frameworks and tooling, not everyday application logic.

Q65 What are annotations, and how are they processed?

ANSWER

Annotations attach metadata to code elements without changing behaviour directly. They are read at compile time (e.g. `@Override`), by tools, or at runtime via reflection (if marked `@Retention(RUNTIME)`). Frameworks use runtime annotations extensively — `@Autowired`, `@Entity`, `@Test` — to drive behaviour declaratively.

© 2026 ByteMart Digital

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Audited { }   // a custom runtime annotation
```

Q66 What is the order of initialization in a Java class?

ANSWER

When a class loads and an object is created, initialisation follows a fixed order: static fields and static initialiser blocks run once when the class first loads (parent before child), then for each instance the instance fields and instance initialiser blocks run, and finally the constructor body — again parent before child via the implicit `super()` call.

SECTION 12

Coding Challenges

Q67 Reverse a string in place without StringBuilder.reverse().

</> CODE

SOLUTION

Convert to a char array and swap characters from both ends toward the middle. This is O(n) time and O(n) space for the array (Java strings are immutable, so some copy is unavoidable).

java

```
static String reverse(String s) {
    char[] c = s.toCharArray();
    int i = 0, j = c.length - 1;
    while (i < j) {
        char t = c[i]; c[i] = c[j]; c[j] = t;
        i++; j--;
    }
    return new String(c);
}
```

Q68 Check whether a string is a palindrome.

</> CODE

SOLUTION

Compare characters from the two ends moving inward; if any pair differs it is not a palindrome. O(n) time, O(1) extra space — no reversal or extra array needed.

java

```
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}
```

NOTE

For a case- and space-insensitive check, normalise first: `s = s.toLowerCase().replaceAll("[^a-z0-9]", "")`.

Q69 Find the first non-repeating character in a string.

</> CODE

SOLUTION

Count occurrences in one pass using a `LinkedHashMap`, which preserves insertion order, then return the first character whose count is one. O(n) time.

```java
static Character firstUnique(String s) {
    Map<Character, Integer> counts = new LinkedHashMap<>();
    for (char ch : s.toCharArray())
        counts.merge(ch, 1, Integer::sum);
    for (var e : counts.entrySet())
        if (e.getValue() == 1) return e.getKey();
    return null;
}
```

Q70 Solve Two Sum — return indices of two numbers adding to a target.

</> CODE

SOLUTION

Keep a map from value to index. For each number, check whether its complement (`target - num`) has already been seen. This turns the naive O(n^2) double loop into a single O(n) pass.

```java
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need))
            return new int[]{ seen.get(need), i };
        seen.put(nums[i], i);
    }
    return new int[0];
}
```

Q71 Reverse a singly linked list.

</> CODE

SOLUTION

Walk the list once, re-pointing each node's `next` to its predecessor while tracking the previous node. O(n) time, O(1) space. The old tail becomes the new head.

```java
static Node reverse(Node head) {
    Node prev = null, curr = head;
    while (curr != null) {
        Node next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;   // new head
}
```

Q72 Find all duplicate elements in an array.

</> CODE

SOLUTION

Track seen values in a `HashSet`; `add` returns false when the value is already present, marking a duplicate. O(n) time, O(n) space.

```java
static Set<Integer> duplicates(int[] nums) {
    Set<Integer> seen = new HashSet<>(), dups = new HashSet<>();
    for (int n : nums)
        if (!seen.add(n)) dups.add(n);
    return dups;
}
```

Q73 Count word frequency in a sentence using the Stream API.

</> CODE

SOLUTION

Split into words, then group and count with `Collectors.groupingBy` and `counting` — a concise, declarative solution that reads almost like the problem statement.

```java
static Map<String, Long> wordFrequency(String text) {
    return Arrays.stream(text.toLowerCase().split("\\s+"))
        .filter(w -> !w.isBlank())
        .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
}
```

© 2026 ByteMart Digital

Q74 Check whether two strings are anagrams.

</> CODE

SOLUTION

Anagrams have identical character counts. For lowercase letters, a fixed-size count array gives an O(n) solution: increment for one string, decrement for the other, and verify all counts return to zero.

java

```java
static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        count[a.charAt(i) - 'a']++;
        count[b.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

Q75 Compute the nth Fibonacci number efficiently.

</> CODE

SOLUTION

The naive recursion is exponential because it recomputes subproblems. An iterative bottom-up approach keeps only the last two values, giving O(n) time and O(1) space.

java

```java
static long fib(int n) {
    if (n < 2) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        long next = a + b;
        a = b; b = next;
    }
    return b;
}
```

NOTE

If you must use recursion, add memoization (an array or map cache) to avoid recomputing the same subproblems.

Q76 Find the missing number in an array of 1..n.

</> CODE

SOLUTION

The sum of 1 to n is `n(n+1)/2`. Subtract the actual array sum from the expected sum to get the missing value — O(n) time, O(1) space, and no sorting.

java

```java
static int missing(int[] nums, int n) {
    long expected = (long) n * (n + 1) / 2;
    long actual = 0;
    for (int x : nums) actual += x;
    return (int) (expected - actual);
}
```